LAPORAN PRATIKUM
BIG-O, SPACE & TIME COMPLEXITY

DISUSUN OLEH :
SITI NASYWAUL
2025573010027

Kelas	:TI 1C
Mata Kuliah	:Algoritma dan Struktur Data
Jurusan	:Teknologi Informasi dan Komputer
Prodi	:Teknik Informatika
Dosen	:Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025-2026

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum Algoritma dan Struktur Data ini dengan baik dan tepat pada waktunya.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan, khususnya pada materi mengenai kompleksitas algoritma, Big-O notation, space complexity, serta teknik refactoring dalam pemrograman JavaScript. Melalui praktikum ini, saya memperoleh pemahaman yang lebih mendalam mengenai pentingnya efisiensi dalam penulisan algoritma, baik dari segi waktu eksekusi maupun penggunaan memori.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi isi maupun penyajian. Hal ini dikarenakan keterbatasan pengetahuan dan pengalaman yang saya miliki. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Saya juga mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan dan arahan selama proses praktikum berlangsung, sehingga laporan ini dapat terselesaikan dengan baik.

Akhir kata, saya berharap laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai konsep dasar algoritma dan efisiensi dalam pemrograman.

Lhokseumawe , 22 April 2026

Siti Nasywaul

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 2
BAB II PEMBAHASAN 4
2.1 Intro Kompleksitas (`01-intro-kompleksitas.js`) 4
2.2 Kelas Big-O dan Destructuring Spread (`02-kelas-big-o.js`) 6
2.3 Space Complexity (`03-space complexity.js`) 9
2.4 Refactor (`04-refactor.js`) 11
2.5 Latihan-1 15
2.6 Tugas 1 17
2.7 Tugas 2 21
BAB III PENUTUP 25
3.1 Kesimpulan 25
3.2 Saran 26

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut para programmer untuk mampu menulis kode yang tidak hanya benar secara logika, tetapi juga efisien dalam hal waktu eksekusi dan penggunaan memori. Dalam pengembangan perangkat lunak, sebuah program yang berjalan lambat atau menggunakan sumber daya yang berlebihan dapat menurunkan performa sistem secara keseluruhan dan berdampak pada pengalaman pengguna.

Salah satu faktor utama yang memengaruhi efisiensi program adalah algoritma yang digunakan. Algoritma yang kurang optimal dapat menyebabkan waktu proses menjadi lebih lama, terutama ketika menangani data dalam jumlah besar. Oleh karena itu, diperlukan pemahaman yang baik mengenai kompleksitas algoritma, baik kompleksitas waktu (time complexity) maupun kompleksitas ruang (space complexity).

Dalam ilmu komputer, analisis kompleksitas algoritma umumnya dinyatakan menggunakan notasi Big-O. Notasi ini membantu dalam mengukur seberapa cepat atau lambat suatu algoritma berjalan seiring dengan bertambahnya ukuran input. Dengan memahami Big-O, seorang programmer dapat membandingkan beberapa algoritma dan memilih solusi yang paling efisien.

Selain itu, penggunaan memori juga menjadi aspek penting yang perlu diperhatikan. Space complexity berkaitan dengan jumlah memori yang dibutuhkan oleh suatu program selama proses eksekusi. Program yang menggunakan memori secara berlebihan dapat menyebabkan pemborosan sumber daya, bahkan dapat mengakibatkan kegagalan sistem pada perangkat dengan kapasitas terbatas.

Tidak hanya itu, kualitas kode juga menjadi perhatian dalam pengembangan perangkat lunak. Kode yang tidak terstruktur dengan baik akan sulit dibaca, dipahami, dan dikembangkan lebih lanjut. Oleh karena itu, diperlukan teknik refactoring, yaitu proses memperbaiki struktur internal kode tanpa mengubah fungsionalitasnya. Refactoring bertujuan untuk meningkatkan keterbacaan, efisiensi, serta kemudahan dalam pemeliharaan kode.

Pada praktikum ini, mahasiswa mempelajari berbagai konsep penting terkait efisiensi algoritma melalui beberapa file JavaScript, seperti pengenalan kompleksitas, penerapan Big-O, analisis penggunaan memori, serta teknik refactoring kode. Dengan adanya praktikum ini, diharapkan mahasiswa mampu memahami dan menerapkan konsep-konsep tersebut dalam pembuatan program yang lebih optimal, efisien, dan mudah dikembangkan di masa mendatang.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Untuk memahami konsep dasar algoritma dan pentingnya efisiensi dalam pemrograman.
2. Untuk mengetahui dan menganalisis kompleksitas waktu (time complexity) dari suatu algoritma menggunakan notasi Big-O.
3. Untuk memahami berbagai jenis kompleksitas algoritma seperti $O(1)$, $O(n)$, dan $O(n^2)$ serta penerapannya dalam program.
4. Untuk menganalisis penggunaan memori dalam suatu program melalui konsep space complexity.
5. Untuk meningkatkan kemampuan dalam menulis kode program yang lebih efisien, terstruktur, dan mudah dipahami.
6. Untuk memahami dan menerapkan teknik refactoring guna memperbaiki kualitas kode tanpa mengubah fungsi utama program.
7. Untuk melatih kemampuan dalam menyelesaikan permasalahan menggunakan algoritma yang optimal.
8. Untuk membiasakan diri dalam menulis kode yang rapi, efektif, dan sesuai dengan kaidah pemrograman yang baik.
9. Untuk mengimplementasikan konsep-konsep yang telah dipelajari ke dalam file praktikum berbasis JavaScript.
10. Untuk meningkatkan keterampilan analisis dalam membandingkan beberapa solusi algoritma berdasarkan tingkat efisiensinya.

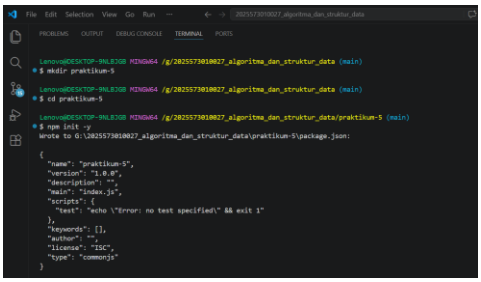
1. Langkah mkdir praktikum-5

- **Perintah:** mkdir praktikum-5
- **Fungsi:** Untuk **membuat folder baru**.
- **Artinya:** Pada langkah ini dibuat folder bernama **praktikum-5** sebagai tempat menyimpan seluruh file program untuk materi Into kompleksitas, Kelas big-o, Space complexity, dan Refactor.

---

2. Langkah cd praktikum-5

- **Perintah:** cd praktikum-5
- **Fungsi:** Untuk **masuk ke dalam folder** yang sudah dibuat.
- **Artinya:** Terminal berpindah ke direktori **praktikum-5**, sehingga semua file yang dibuat setelahnya akan tersimpan di folder tersebut.



---

3. Langkah npm init -y

- **Perintah:** npm init -y
- **Fungsi:** Untuk **membuat project Node.js secara otomatis**.
- **Artinya:** Sistem akan langsung membuat file **package.json** dengan konfigurasi default tanpa harus mengisi data secara manual. File ini berfungsi sebagai identitas project dan tempat menyimpan informasi dasar project JavaScript.

---

4. Tampilan File package.json

- Setelah perintah dijalankan, akan muncul file **package.json** di sidebar VS Code.
- File ini berisi informasi seperti:
  - **name** → nama project
  - **version** → versi project
  - **main** → file utama
  - **scripts** → script command
- **Artinya:** Project JavaScript sudah siap digunakan untuk membuat file praktikum seperti:
  - 01- intro-kompleksitas.js
  - 02-kelas-big-o.js
  - 03-space complexity.js
  - 04-refactor.js
  - latihan-1.js

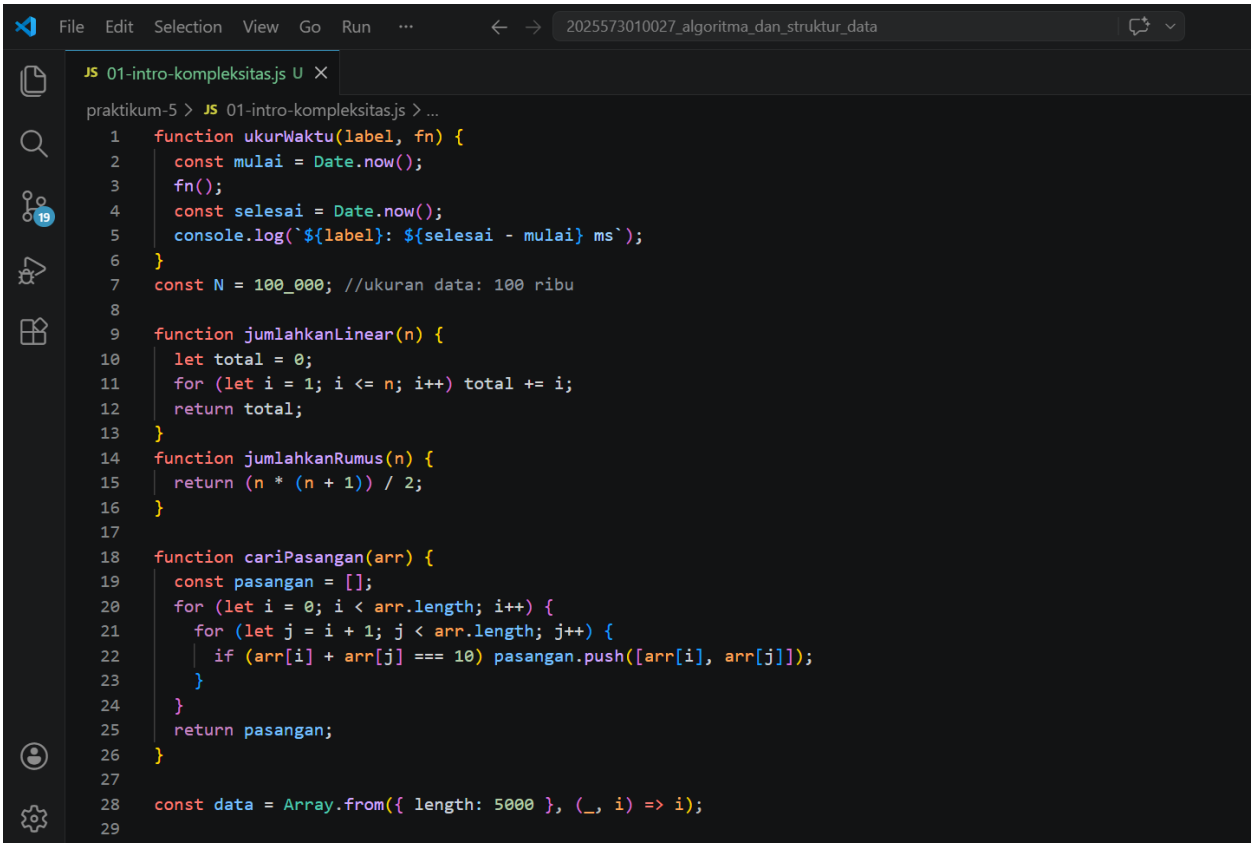
BAB II PEMBAHASAN

2.1 Intro Kompleksitas (`01-intro-kompleksitas.js`)

Pada bagian ini diperkenalkan konsep dasar kompleksitas algoritma. Kompleksitas algoritma merupakan ukuran yang digunakan untuk menentukan tingkat efisiensi suatu algoritma dalam menyelesaikan masalah. Efisiensi ini dilihat dari dua aspek utama, yaitu waktu eksekusi (time complexity) dan penggunaan memori (space complexity).

Dalam file ini biasanya ditampilkan contoh sederhana seperti penggunaan perulangan (loop). Misalnya, sebuah perulangan tunggal yang berjalan dari awal hingga akhir data memiliki kompleksitas O(n), karena waktu yang dibutuhkan bergantung pada jumlah data yang diproses.

Selain itu, diperkenalkan juga bahwa semakin banyak perulangan bersarang (nested loop), maka kompleksitas algoritma akan semakin besar, seperti $O(n^2)$. Dengan memahami konsep ini, mahasiswa dapat mulai mengenali perbedaan antara algoritma yang efisien dan yang tidak efisien.




```js
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; //ukuran data: 100 ribu

function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}

const data = Array.from({ length: 5000 }, (_, i) => i);
```

```javascript
console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("0(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("0(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("0(n^2) cariPasangan ", () => cariPasangan(data));

console.log("/nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));

```

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
0(1) jumlahkanRumus : 0 ms
0(n) jumlahkanLinear : 3 ms
0(n^2) cariPasangan : 69 ms

Hasil sama? true

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

2.2 Kelas Big-O dan Destructuring Spread (`02-kelas-big-o.js`)

Pada bagian ini dibahas lebih dalam mengenai notasi Big-O sebagai alat untuk mengukur kompleksitas waktu suatu algoritma. Big-O digunakan untuk menggambarkan seberapa besar pertambahan waktu eksekusi program terhadap pertambahan jumlah input.

Beberapa jenis kompleksitas yang umum dijelaskan antara lain:

- O(1) → waktu konstan
- O(n) → waktu linear
- O(n^2) → waktu kuadratik

Selain itu, pada file ini juga diterapkan konsep **destructuring** dan **spread operator** dalam JavaScript. Destructuring digunakan untuk mengambil nilai dari array atau object secara langsung ke dalam variabel, sehingga kode menjadi lebih ringkas dan mudah dibaca.

Contoh sederhana:

</> JavaScript

```javascript
const data = { nama: "Nasywa", umur: 20 };

const { nama, umur } = data;
```

Sedangkan spread operator (...) digunakan untuk menyalin atau menggabungkan data, seperti array atau object, tanpa mengubah data aslinya.

Contoh:

</> JavaScript

```javascript
const angka1 = [1, 2];

const angka2 = [3, 4];

const gabung = [...angka1, ...angka2];
```

Penggunaan teknik ini membantu dalam penulisan kode yang lebih efisien dan modern, serta mempermudah proses manipulasi data.

File Edit Selection View Go Run ··· 2025573010027_algoritma_dan_struktur_data

02-destructuring-spread.js U

praktikum-5 > 02-destructuring-spread.js > ...

```javascript
function ambilPertama(arr) {
  return arr[0];
}
function tambahkanItem(arr, item) {
  arr.push(item);
}
function isGenap(n) {
  return n % 2 === 0;
}

function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(`Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}

function cariMax(arr) {
  let maks = arr[0];
```

```javascript
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}

function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length - 1; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}

function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2);
}

console.log("=== O(1) - selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42));

console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data

console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
```

```javascript
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);

console.log("\n=== O(n^2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log("\n=== O(2^n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");

console.log("\nWaktu fib(35) O(2^n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");

console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 02-destructuring-spread.js
=== 0(1) - selalu cepat ===
10
true

=== O(log n) — Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n^2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
213 ms
Waktu fib(35) O(n) memoization:
1 ms
```

2.3 Space Complexity (`03-space complexity.js`)

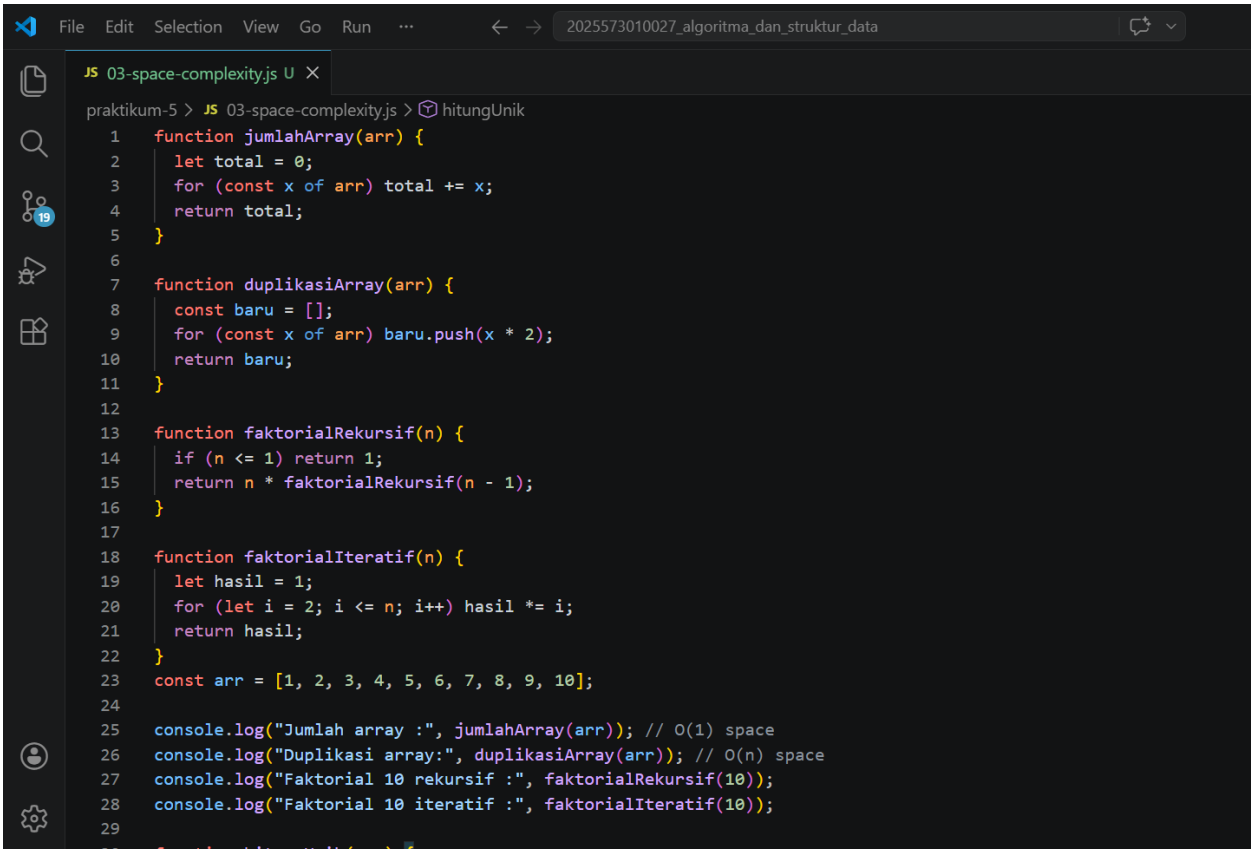
Pada bagian ini dibahas mengenai kompleksitas ruang (space complexity), yaitu ukuran jumlah memori yang digunakan oleh suatu algoritma selama proses eksekusi.

Space complexity sangat penting karena penggunaan memori yang berlebihan dapat menyebabkan program menjadi tidak efisien, terutama pada perangkat dengan keterbatasan sumber daya.

Contoh sederhana:

- Penggunaan variabel biasa → memori kecil (O(1))
- Penggunaan array tambahan → memori bertambah sesuai jumlah data (O(n))

Melalui file ini, mahasiswa dapat memahami bahwa tidak hanya kecepatan program yang perlu diperhatikan, tetapi juga efisiensi penggunaan memori. Dengan demikian, programmer dapat membuat program yang lebih optimal secara keseluruhan.




```js
function jumlahArray(arr) {
  let total = 0;
  for (const x of arr) total += x;
  return total;
}

function duplikasiArray(arr) {
  const baru = [];
  for (const x of arr) baru.push(x * 2);
  return baru;
}

function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1);
}

function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i;
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
```

```javascript
function hitungUnik(arr) {
  const seen = new Set();
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

2.4 Refactor (`04-refactor.js`)

Pada bagian ini dibahas mengenai refactoring, yaitu proses memperbaiki struktur dan kualitas kode tanpa mengubah fungsi atau output dari program.

Refactoring bertujuan untuk:

- Membuat kode lebih rapi dan terstruktur
- Meningkatkan keterbacaan kode
- Mempermudah proses pengembangan dan debugging
- Mengurangi kompleksitas kode

Contoh refactoring dapat berupa:

- Mengganti variabel yang tidak jelas menjadi lebih deskriptif
- Menghapus kode yang tidak digunakan
- Memecah fungsi yang terlalu panjang menjadi beberapa fungsi kecil
- Menggunakan metode yang lebih efisien

Dengan melakukan refactoring, kode program menjadi lebih profesional dan mudah dipahami oleh programmer lain. Hal ini sangat penting dalam pengembangan software secara tim maupun jangka panjang.

File Edit Selection View Go Run ··· 2025573010027_algoritma_dan_struktur_data

04-refactor.js U

praktikum-5 > 04-refactor.js > ...

```javascript
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}

function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
```

```javascript
    return counter;
  }

  const besar = Array.from({ length: 20000 }, () =>
    Math.floor(Math.random() * 1000),
  );

  let t = Date.now();
  adaDuplikatLambat(besar);
  console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");

  t = Date.now();
  adaDuplikatCepat(besar);
  console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");

  const kata = ["a", "b", "a", "c", "b", "a", "d"];
  console.log("\nFrekuensi:", frekuensiCepat(kata));
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                bash - p

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 1 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

Latihan-3

```javascript
console.log("\n=== LATIHAN 3: Cari Pasangan ===");
// Fungsi lambat → O(n^2)
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// Fungsi cepat → O(n)
function cariPasanganCepat(arr, target) {
  let set = new Set();

  for (let num of arr) {
    let pasangan = target - num;

    if (set.has(pasangan)) {
      return [pasangan, num];
    }

    set.add(num);
  }

  return null;
```

```javascript
}

// Test kecil
let arr = [2, 7, 11, 15];
let target = 9;

console.log("Lambat:", cariPasanganLambat(arr, target));
console.log("Cepat :", cariPasanganCepat(arr, target));

// Test waktu besar
function testWaktu(fn, arr, target) {
  let start = Date.now();
  fn(arr, target);
  let end = Date.now();
  console.log(`${fn.name}: ${end - start} ms`);
}

// generate 50.000 data
let bigArr = Array.from({ length: 50000 }, () =>
  Math.floor(Math.random() * 100000),
);

let targetBesar = -1;

console.log("\n=== UJI WAKTU ===");
testWaktu(cariPasanganLambat, bigArr, targetBesar);
testWaktu(cariPasanganCepat, bigArr, targetBesar);
```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }

=== LATIHAN 3: Cari Pasangan ===
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]

=== UJI WAKTU ===
cariPasanganLambat: 5157 ms
cariPasanganCepat: 15 ms

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

2.5 Latihan-1

```
File  Edit  Selection  View  Go  Run  ···   2025573010027_algoritma_dan_struktur_data
JS latihan-1.js U ×
praktikum-5 > JS latihan-1.js > B
```

```javascript
console.log("=== LATIHAN 1: Kompleksitas ===");

// Fungsi A → O(1)
function A(n) {
  return n * 2;
}

// Fungsi B → O(n^2)
function B(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // console.log(i, j);
    }
  }
}

// Fungsi C → O(log n)
function C(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// Fungsi D → O(n^3)
function D(n) {
  let arr = new Array(n).fill(0);
  arr.forEach(() =>
    arr.forEach(() =>
      arr.forEach(() => {
```

```javascript
        // console.log("x");
      }),
    ),
  );
}

// Fungsi ukur waktu
function hitungKompleksitas(n, fn) {
  let start = Date.now();
  fn(n);
  let end = Date.now();
  console.log(`${fn.name}: ${end - start} ms`);
}

// Jalankan Latihan 1
hitungKompleksitas(1000, A);
hitungKompleksitas(1000, B);
hitungKompleksitas(1000, C);
hitungKompleksitas(1000, D);
```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node latihan-1.js
=== LATIHAN 1: Kompleksitas ===
A: 0 ms
B: 5 ms
C: 0 ms
D: 1501 ms

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

2.6 Tugas 1

```javascript
console.log("=== TUGAS 1: Analisis & Refactor ===");

/* ========================
   FUNGSI A: INTERSECTION
   ======================== */

// Versi lambat → O(n^2)
function intersectionLambat(arr1, arr2) {
  let result = [];

  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j]) {
        result.push(arr1[i]);
      }
    }
  }

  return result;
}
/*
Time: O(n^2)
Space: O(n)
*/

// Versi cepat → O(n)
function intersectionCepat(arr1, arr2) {
  let set = new Set(arr2);
  let result = [];

  for (let item of arr1) {
    if (set.has(item)) {
      result.push(item);
    }
  }

  return result;
}
/*
Time: O(n)
Space: O(n)
*/

/* ========================
   FUNGSI B: ANAGRAM GROUP
   ======================== */

// O(n * k log k)
function groupAnagram(arr) {
  let map = new Map();

  for (let word of arr) {
    let key = word.split("").sort().join("");

    if (!map.has(key)) {
      map.set(key, []);
    }
```

```javascript

    map.get(key).push(word);
  }

  return Array.from(map.values());
}
/*
Time: O(n * k log k)
Space: O(n)
*/

/* ========================
   FUNGSI C: 3 SUM
   ======================== */

// Versi lambat → O(n^3)
function threeSumLambat(arr) {
  let n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      for (let k = j + 1; k < n; k++) {
        if (arr[i] + arr[j] === arr[k]) {
          return true;
        }
      }
    }
  }
```

```javascript

  return false;
}
/*
Time: O(n^3)
Space: O(1)
*/

// Versi cepat → O(n log n)
function threeSumCepat(arr) {
  arr.sort((a, b) => a - b); // O(n log n)

  let n = arr.length;

  for (let i = 0; i < n; i++) {
    let left = i + 1;
    let right = n - 1;

    while (left < right) {
      let sum = arr[i] + arr[left];

      if (sum === arr[right]) {
        return true;
      } else if (sum < arr[right]) {
        left++;
      } else {
        right--;
      }
```

```
    }
  }

  return false;
}
/*
Time: O(n log n)
Space: O(1)
*/

/* ========================
   TESTING
   ========================= */

console.log("\n-- Fungsi A --");
console.log(intersectionLambat([1, 2, 3], [2, 3, 4]));
console.log(intersectionCepat([1, 2, 3], [2, 3, 4]));

console.log("\n-- Fungsi B --");
console.log(groupAnagram(["eat", "tea", "tan", "ate", "nat", "bat"]));

console.log("\n-- Fungsi C --");
console.log(threeSumLambat([2, 7, 11, 15])); // true
console.log(threeSumCepat([2, 7, 11, 15])); // true

/* ========================
   UKUR WAKTU
   ========================= */
```

```

function testWaktu(fn, ...args) {
  let start = Date.now();
  fn(...args);
  let end = Date.now();
  console.log(`${fn.name}: ${end - start} ms`);
}

// data besar
let bigArr = Array.from({ length: 10000 }, () =>
  Math.floor(Math.random() * 10000),
);

console.log("\n-- UJI WAKTU --");
testWaktu(threeSumLambat, bigArr);
testWaktu(threeSumCepat, bigArr);

```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ cd praktikum-5/

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node tugas/tugas-1.js
=== TUGAS 1: Analisis & Refactor ===

-- Fungsi A --
[ 2, 3 ]
[ 2, 3 ]

-- Fungsi B --
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

-- Fungsi C --
false
false

-- UJI WAKTU --
threeSumLambat: 19 ms
threeSumCepat: 14 ms
```

2.7 Tugas 2

File Edit Selection View Go Run ··· 2025573010027_algoritma_dan_struktur_data

tugas-2.js U

praktikum-5 > tugas > tugas-2.js > ...

```javascript
console.log("=== TUGAS 2: Visualisasi Big O ===");

/* ========================
   DEFINISI FUNGSI
   ======================== */

// O(1)
function fn_O1(n) {
  return n + 1;
}

// O(n)
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
function fn_OnLogn(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    // log2(n)
    for (let j = 1; j < n; j *= 2) {
      count++;
    }
```

```javascript
  }

  return count;
}

// O(n^2)
function fn_On2(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }

  return count;
}

/* ========================
   BENCHMARK
   ======================== */

function ukur(fn, n) {
  let start = Date.now();
  fn(n);
  let end = Date.now();
  return end - start;
}
```

```javascript

function benchmarkSemua(ukuranData) {
  for (let n of ukuranData) {
    console.log(`\nUkuran n = ${n}`);

    console.log("O(1)     :", ukur(fn_O1, n), "ms");
    console.log("O(n)     :", ukur(fn_On, n), "ms");
    console.log("O(n log n):", ukur(fn_OnLogn, n), "ms");
    console.log("O(n^2)   :", ukur(fn_On2, n), "ms");
  }
}

/* ========================
   JALANKAN
   ======================== */

benchmarkSemua([100, 500, 1000, 5000, 10000]);

/* ========================
   ANALISIS (KOMENTAR)
   ======================== */

/*
Hasil yang diamati:

1. O(1):
   Waktu hampir konstan, tidak terpengaruh n.

2. O(n):
```

```
   Waktu bertambah secara linear seiring n.

3. O(n log n):
   Lebih cepat dari O(n^2), tapi lebih lambat dari O(n).

4. O(n^2):
   Pertumbuhan paling cepat, sangat terasa saat n besar.

Kesimpulan:
Pola pertumbuhan sesuai teori Big O.
Semakin tinggi kompleksitas, semakin cepat waktu eksekusi meningkat.
*/

```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ node tugas/tugas-2.js
=== TUGAS 2: Visualisasi Big O ===

Ukuran n = 100
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 1 ms

Ukuran n = 500
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 1 ms

Ukuran n = 1000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n^2)   : 1 ms

Ukuran n = 5000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 1 ms
O(n^2)   : 31 ms

Ukuran n = 10000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 1 ms
O(n^2)   : 272 ms
```

Semua file yang sudah di kerjakan kita push ke github

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it

Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main f81f0ba] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 571 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

```
Lenovo@DESKTOP-9NLBJGB MINGW64 /g/2025573010027_algoritma_dan_struktur_data/praktikum-5 (main)
$ git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.69 MiB | 368.00 KiB/s, done.
From https://github.com/SitiNasywaul/2025573010027_algoritma_dan_struktur_data
 * branch            main       -> FETCH_HEAD
   eecf9c0..cefeaf4  main       -> origin/main
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced by CRLF the next time Git touches it
hint: Waiting for your editor to close the file...
```

Name	Last commit message	Last commit date
..		
tugas	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
01-intro-kompleksitas.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
02-kelas-big-o.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
03-space-complexity.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
04-refactor.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
latihan-1.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
package.json	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago

Name	Last commit message	Last commit date
..		
tugas-1.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago
tugas-2.js	Praktikum 5: Analisis Kompleksitas Algoritma	7 hours ago

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan pada praktikum ke-5 mengenai kompleksitas algoritma, dapat disimpulkan bahwa pemahaman terhadap efisiensi algoritma merupakan hal yang sangat penting dalam pengembangan perangkat lunak. Algoritma yang baik tidak hanya mampu menghasilkan output yang benar, tetapi juga harus mampu bekerja secara optimal dalam hal waktu eksekusi dan penggunaan memori.

Melalui praktikum ini, dapat dipahami bahwa notasi Big-O merupakan alat yang sangat berguna untuk menganalisis dan membandingkan performa suatu algoritma. Dengan menggunakan Big-O, programmer dapat menentukan algoritma mana yang lebih efisien ketika dijalankan pada data dengan ukuran besar. Selain itu, konsep space complexity juga memberikan gambaran mengenai seberapa besar memori yang digunakan oleh suatu program, sehingga dapat membantu dalam menghindari pemborosan sumber daya.

Praktikum ini juga menekankan pentingnya penulisan kode yang rapi dan terstruktur melalui teknik refactoring. Dengan melakukan refactoring, kode menjadi lebih mudah dibaca, dipahami, serta lebih mudah untuk dikembangkan di masa mendatang tanpa mengubah fungsi utama dari program tersebut.

Selain itu, melalui latihan dan tugas yang diberikan, mahasiswa dapat mengasah kemampuan dalam menganalisis permasalahan, memilih algoritma yang tepat, serta mengimplementasikannya ke dalam bahasa pemrograman JavaScript. Hal ini menjadi bekal penting dalam menghadapi permasalahan nyata di dunia pemrograman.

Dengan demikian, praktikum ini memberikan pemahaman yang komprehensif mengenai pentingnya efisiensi algoritma, baik dari segi waktu maupun ruang, serta pentingnya menjaga kualitas kode dalam proses pengembangan perangkat lunak.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan pelaksanaan praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan dapat lebih sering berlatih dalam menganalisis kompleksitas algoritma agar semakin terbiasa dalam menentukan solusi yang efisien.
2. Dalam penulisan kode, sebaiknya mahasiswa membiasakan diri untuk menulis kode yang rapi, terstruktur, dan mudah dibaca sejak awal.
3. Perlu dilakukan eksplorasi lebih lanjut terhadap berbagai jenis algoritma dan teknik optimasi lainnya untuk menambah wawasan.
4. Mahasiswa disarankan untuk tidak hanya fokus pada hasil akhir program, tetapi juga memperhatikan efisiensi dan kualitas kode yang dibuat.
5. Praktikum selanjutnya diharapkan dapat memberikan studi kasus yang lebih kompleks agar kemampuan analisis mahasiswa semakin berkembang.
6. Dosen atau asisten praktikum diharapkan dapat memberikan lebih banyak contoh penerapan nyata agar mahasiswa lebih mudah memahami konsep yang diajarkan.